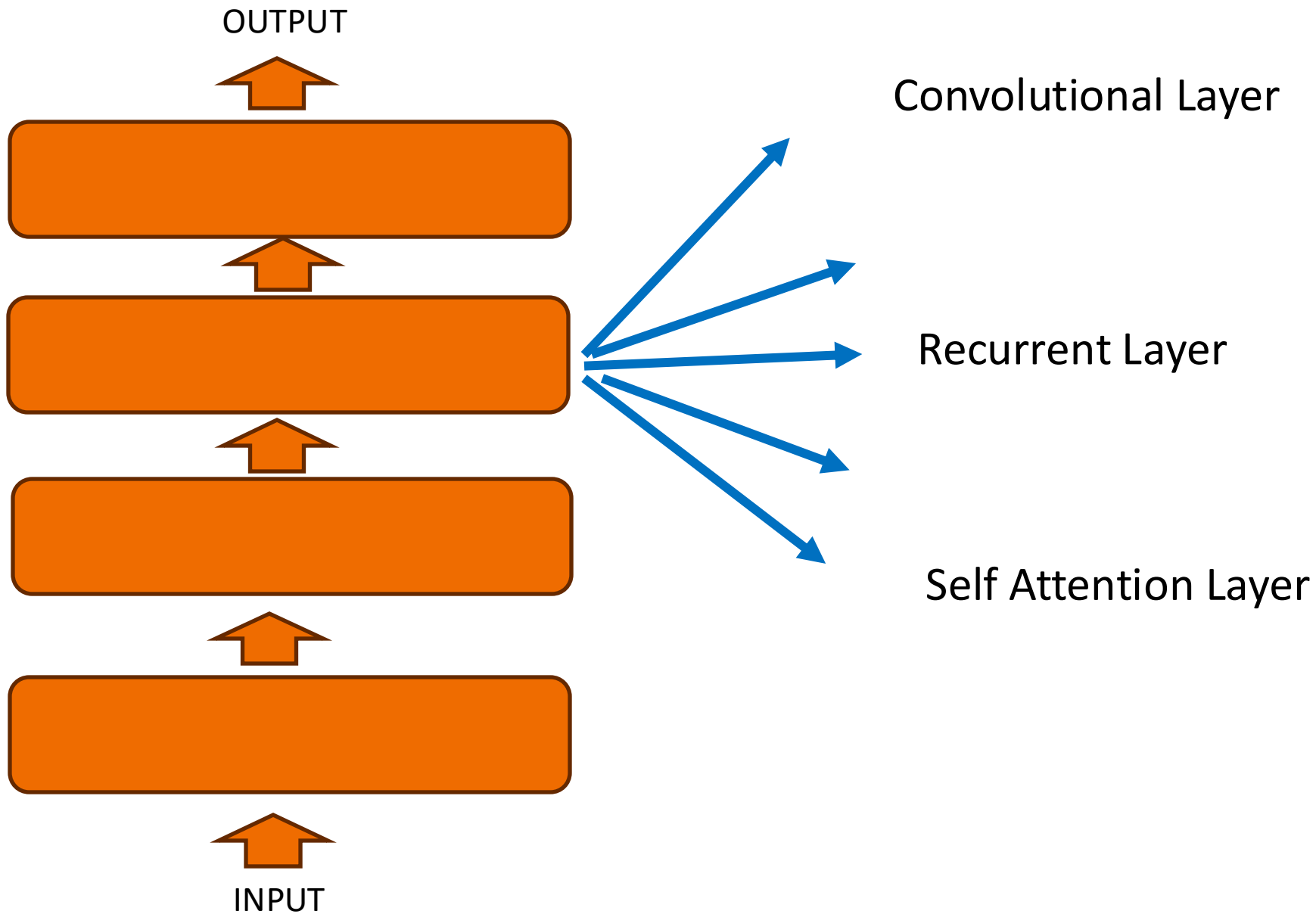
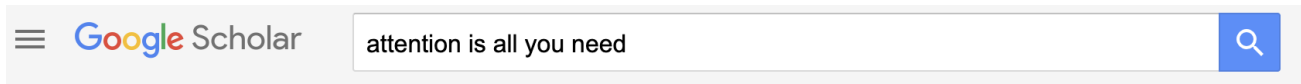


Self-Attention

DL and Different Types of Layers



Transformers



Any time

Since 2024

Since 2023

Since 2020

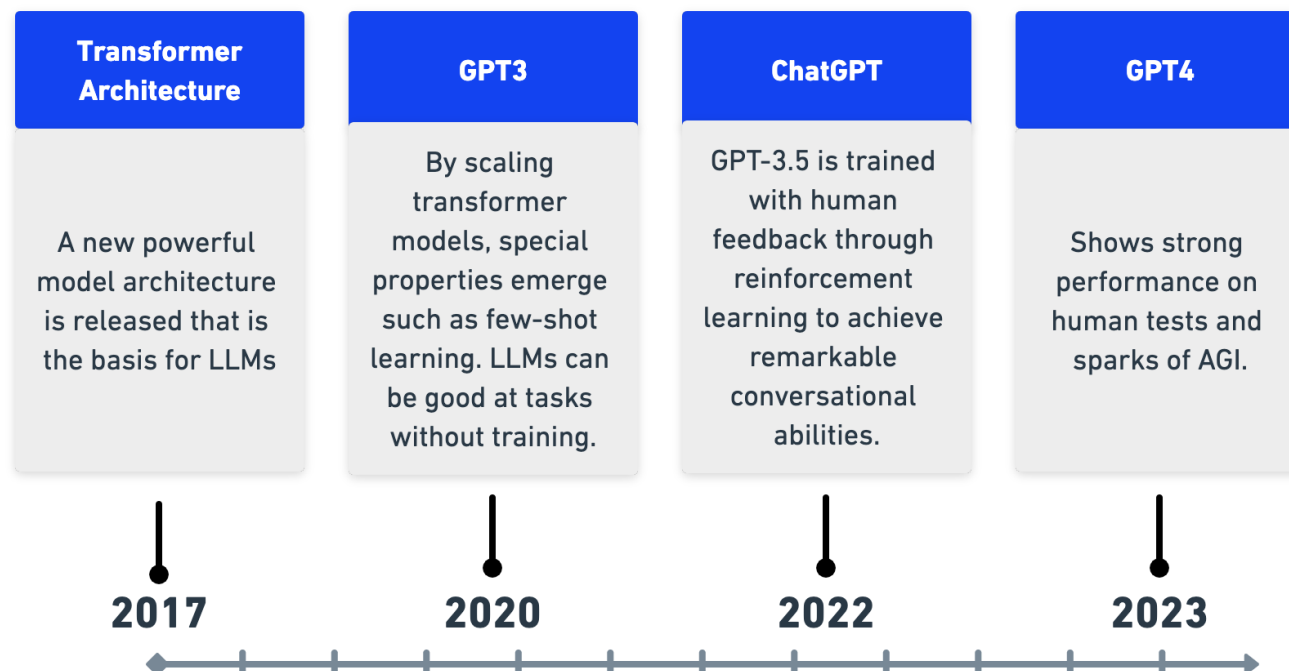
Custom range...

[PDF] **Attention is all you need**

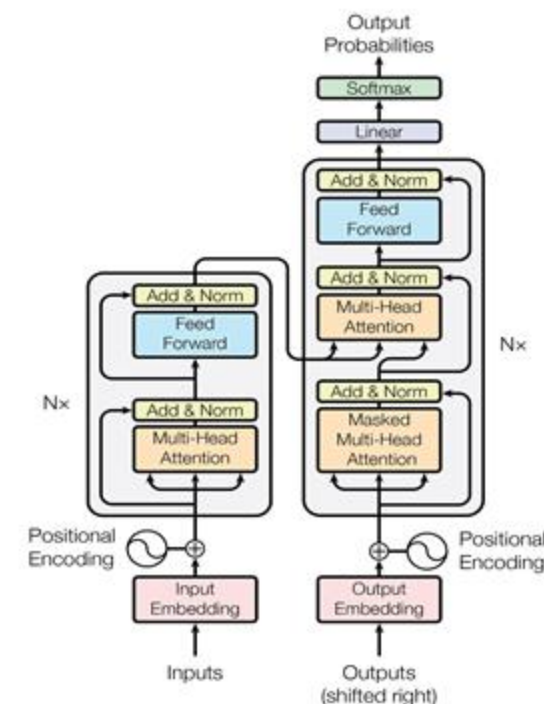
A Vaswani - Advances in Neural Information Processing Systems, 2017 - user.phil.hhu.de

Attention is all you need Attention is all you need ...

☆ Save ↗ Cite Cited by 135180 Related articles ↗



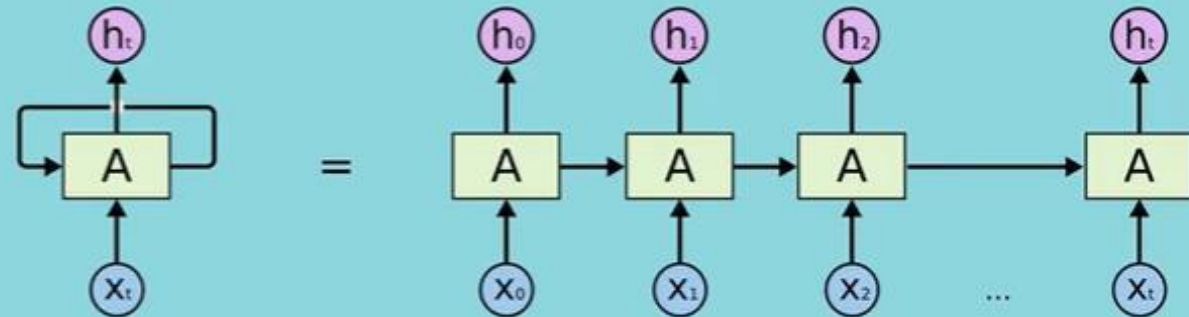
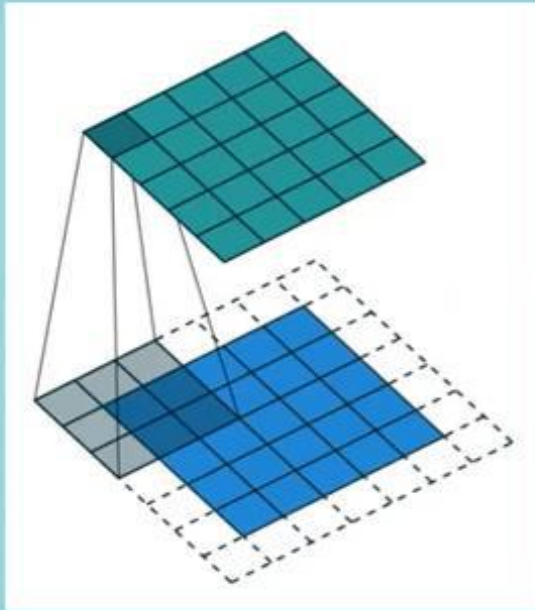
Task: Translate from one language to another



Transformer Encoder
Transformer Decoder

Attention is all you need (2017)

CNN, RNN and Transformers



Pro: Compute in Parallel

Con: Finite Horizon

Pro: Infinite Horizon

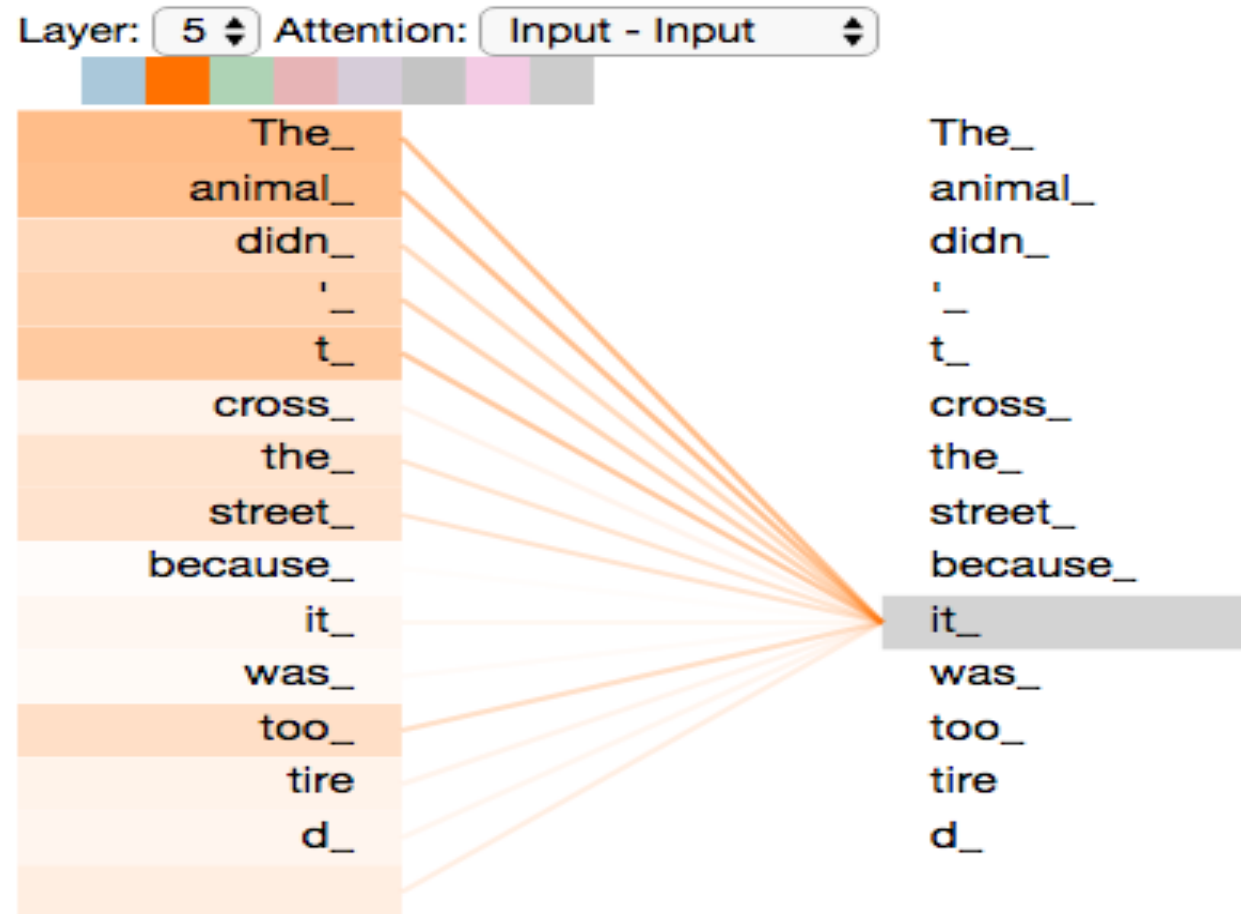
Con: Compute Serially only

Attention

Blank Slide

I. Simple Self Attention

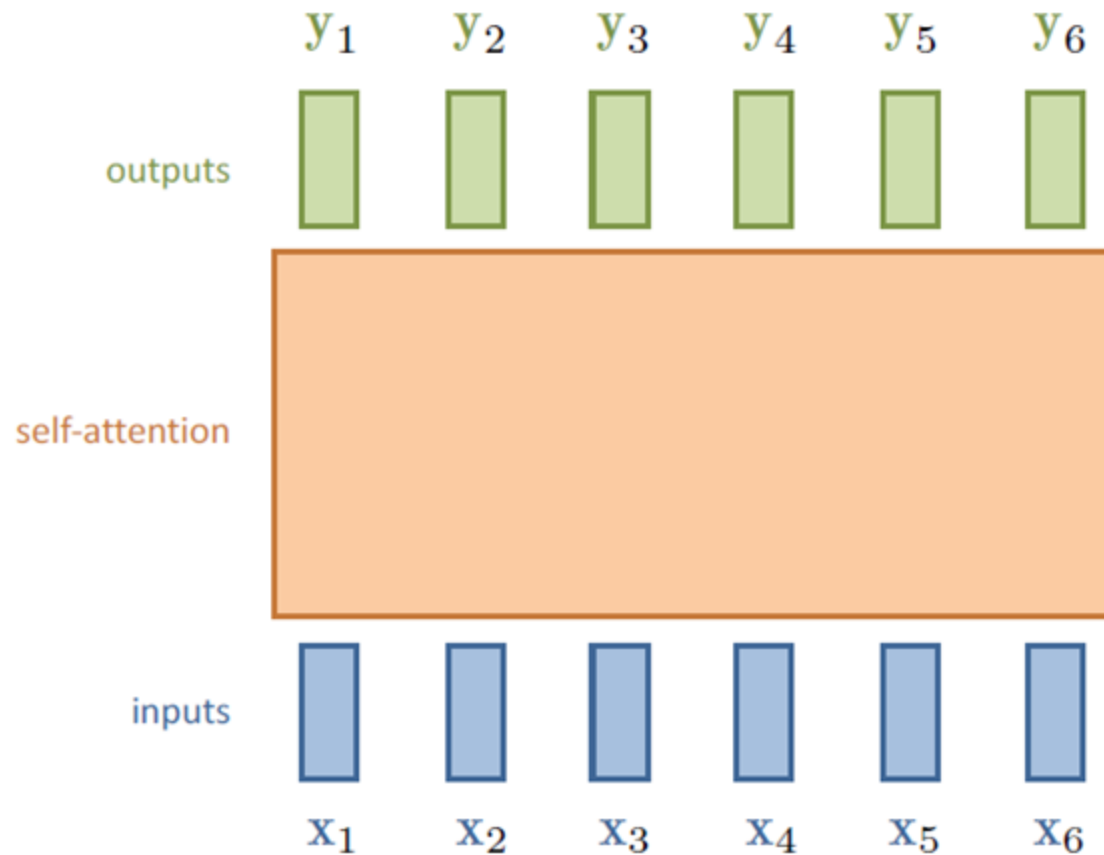
Self Attention



1. The **animal** did not cross the road because **it** was too tired.
2. The animal did not cross **the road** because **it** was too wide.

SA: Computation

SELF-ATTENTION



$$y_i = \sum_j w_{ij} x_j$$

$$y_i = \sum_j w_{ij} x_j$$

$$w_{ij}^0 = x_i^T x_j$$

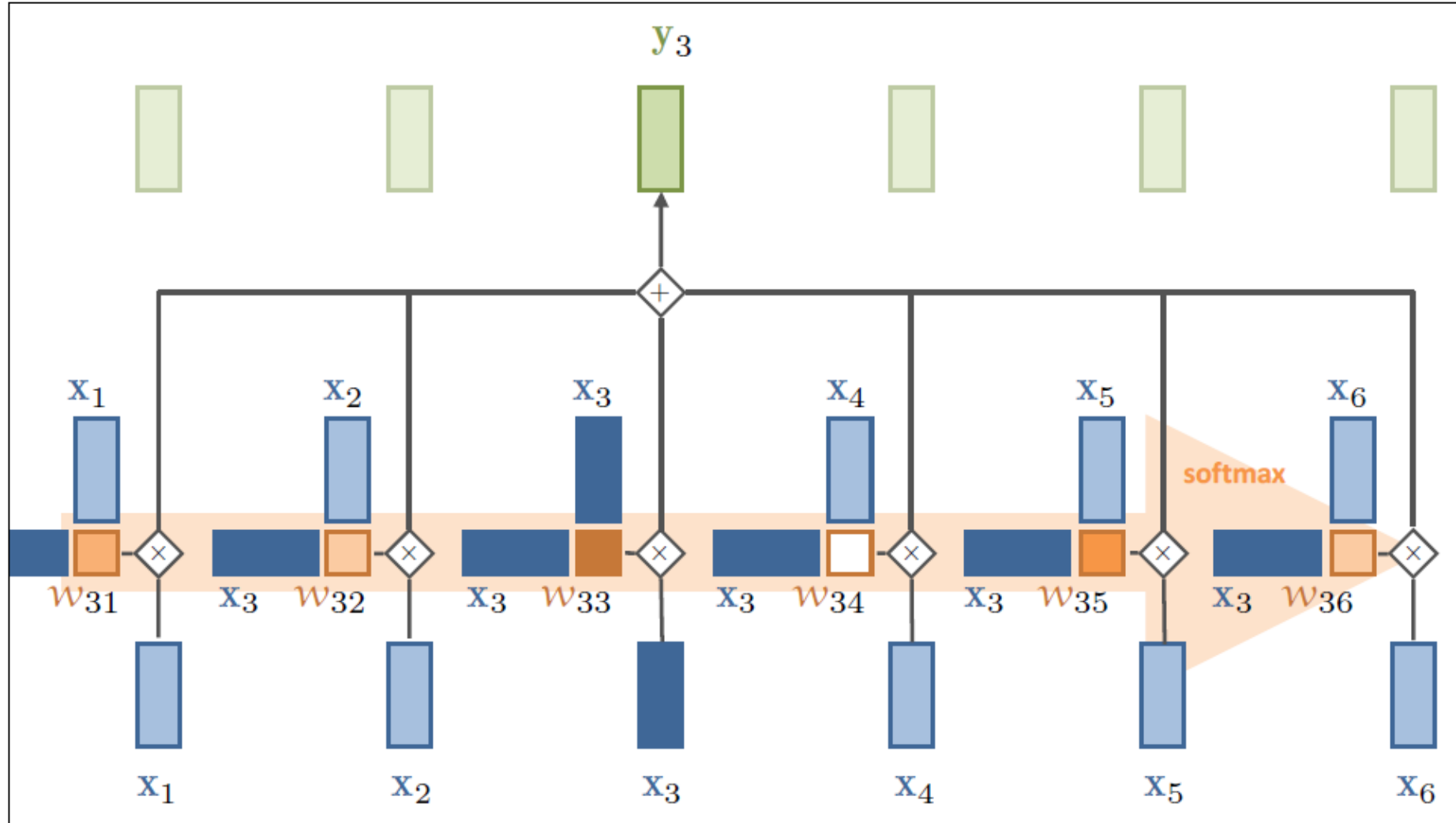
$$w_{ij} = \frac{\exp w_{ij}^0}{\sum_l \exp w_{il}^0}$$

Blank Slide

SA: Observations

- Input and Output:
 - Vectors are of same in number and dimension
 - Set of vectors (in simple form). Add position, if required.
- Every output vector is influenced by “all” input vectors.
- “W” is not learned here. It is computed from the input/data.

SA: Computation



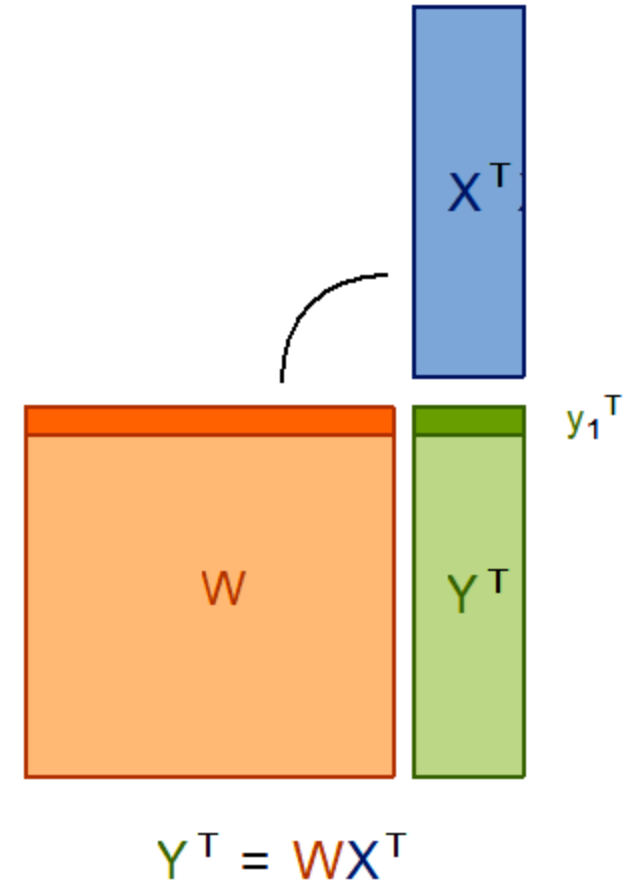
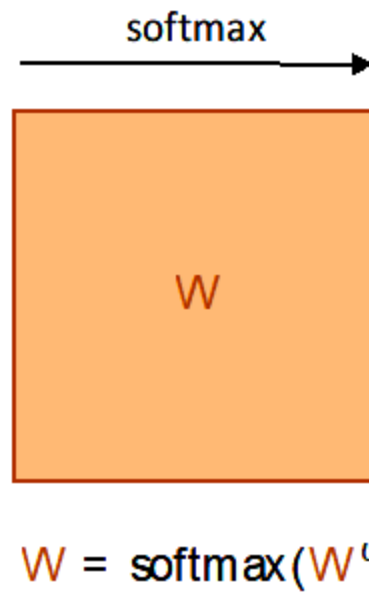
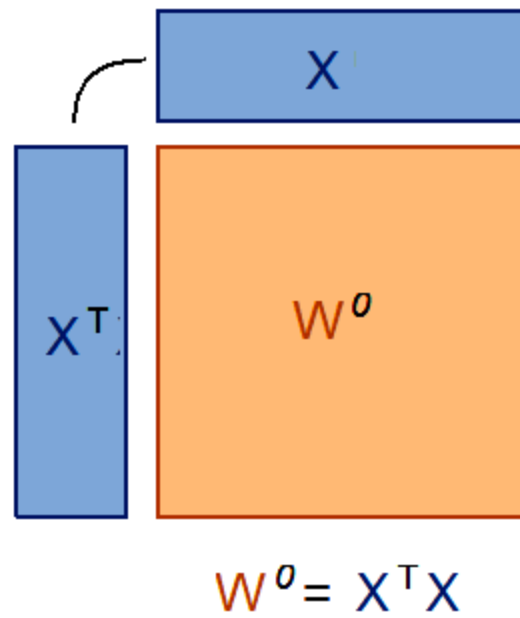
$$y_i = \sum_j w_{ij} x_j$$

$$w_{ij}^0 = x_i^T x_j$$

$$w_{ij} = \frac{\exp w_{ij}^0}{\sum_l \exp w_{il}^0}$$

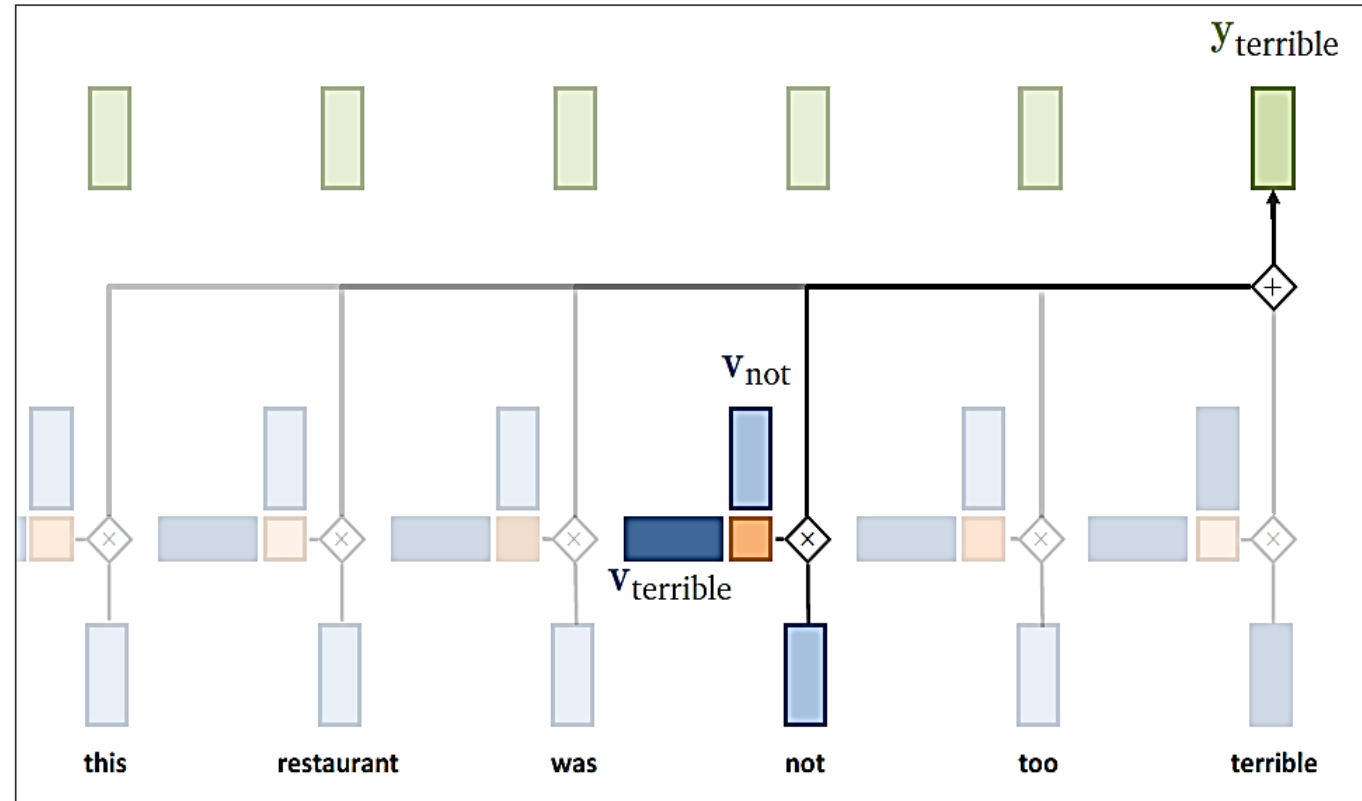
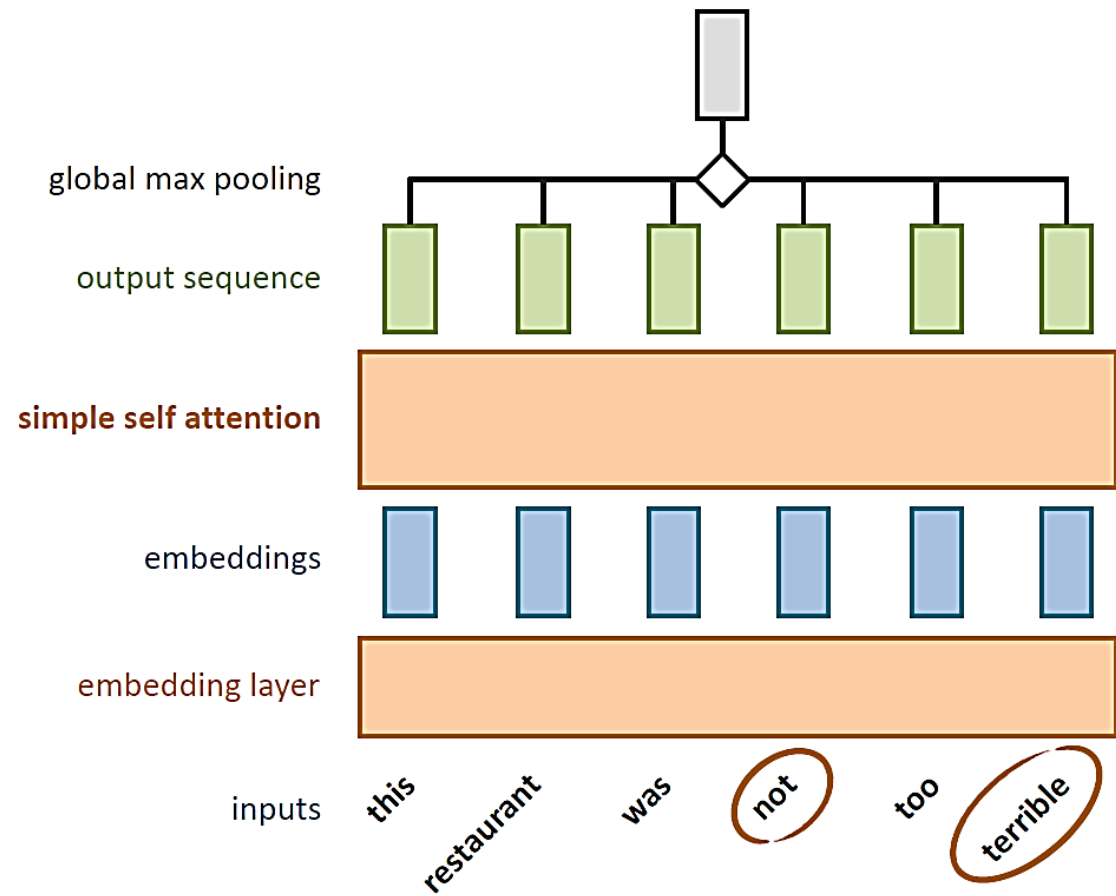
As matrix operations

VECTORIZED



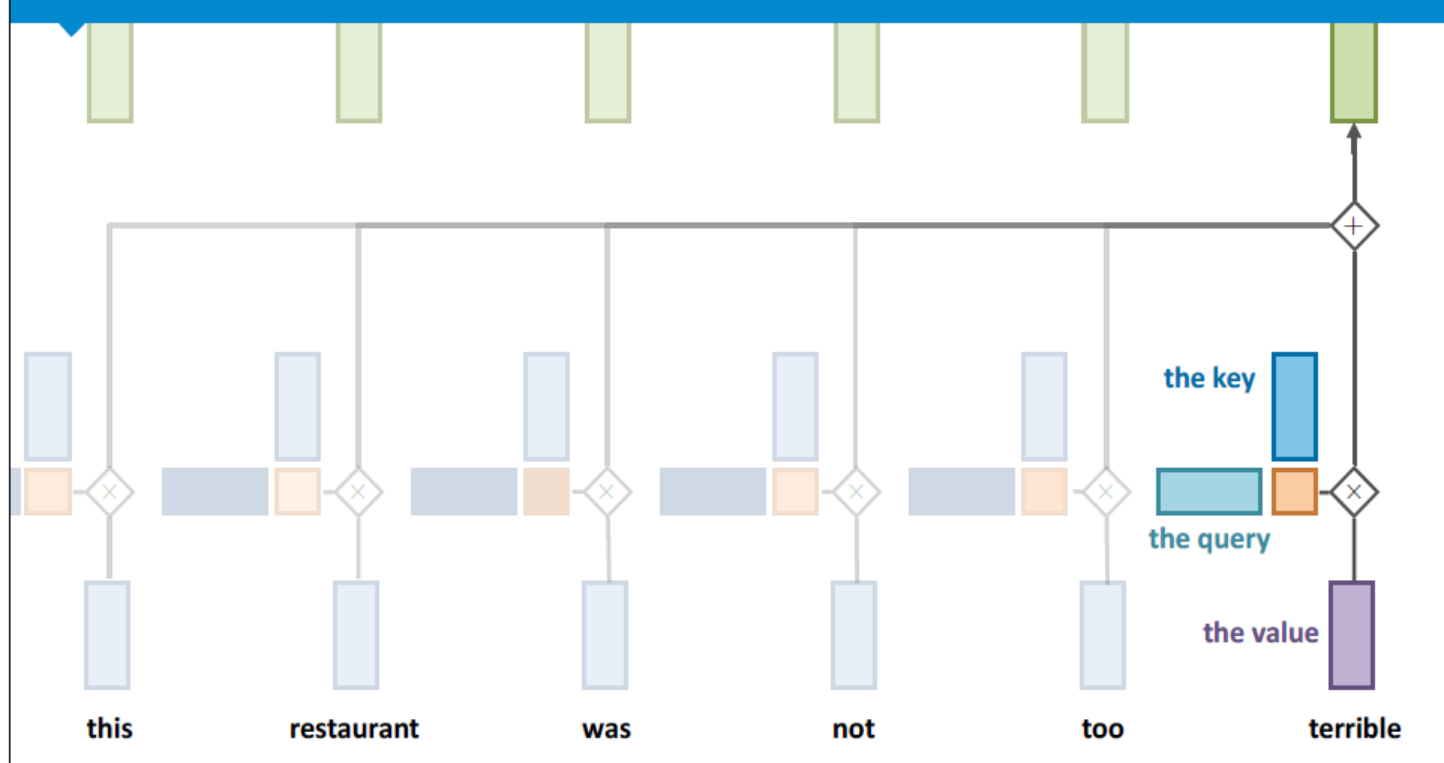
Blank Slide

Example



Query, Key and Value

KEYS, QUERIES AND VALUES



In each self attention computation, every input vector occurs in three distinct roles:

- **the value**: the vector that is used in the weighted sum that ultimately provides the output
- **the query**: the input vector that corresponds to the current output, matched against every other input vector.
- **the key**: the input vector that the query is matched against to determine the weight.

Blank Slide

Example

We ask: what is the value of $\begin{bmatrix} 2 \\ 0 \\ 3 \end{bmatrix}$? $y = \sum_{j=1}^N (k_j^T q) \cdot v_j$

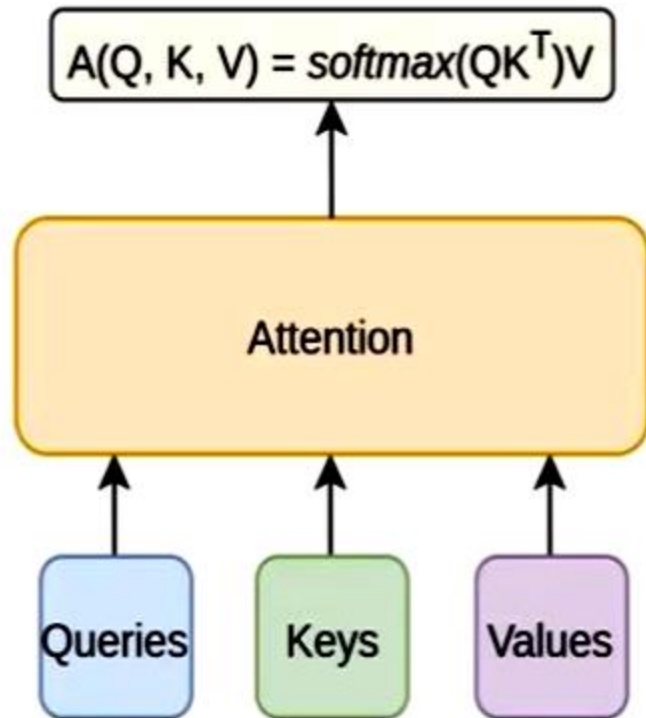
keys	values	weight
$\begin{bmatrix} 0.5 \\ 0.5 \\ 0 \end{bmatrix}$	1	$k_1^T \cdot q = [0.5, 0.5, 0] \cdot \begin{bmatrix} 2 \\ 0 \\ 3 \end{bmatrix} = 0.5 * 2 + 0.5 * 0 + 0 * 3 = 1$
$\begin{bmatrix} 0.5 \\ -0.5 \\ 0 \end{bmatrix}$	3	$k_2^T \cdot q = [0.5, -0.5, 0] \cdot \begin{bmatrix} 2 \\ 0 \\ 3 \end{bmatrix} = 0.5 * 2 + (-0.5) * 0 + 0 * 3 = 1$
$\begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$	5	$k_3^T \cdot q = [0, 0, 1] \cdot \begin{bmatrix} 2 \\ 0 \\ 3 \end{bmatrix} = 0 * 2 + 0 * 0 + 1 * 3 = 3$

$= 1 * 1 + 1 * 3 + 3 * 5 = 19$



Blank Slide

What is Attention?



$$y_i = \sum_{j=1}^N (k_j^T q_i) \cdot v_j$$

$$\text{softmax} \left(\begin{matrix} [Q] \times d_k \\ \text{[Matrix]} \end{matrix} \times \begin{matrix} d_k \times [K] \\ \text{[Matrix]} \end{matrix} \right) \times \begin{matrix} [V] \times d_v \\ \text{[Matrix]} \end{matrix} = [Q] \times d_v$$

The diagram shows the matrix operations for the Attention mechanism. It starts with a softmax function applied to the product of a Queries matrix (size $[Q] \times d_k$, represented by a blue box with horizontal lines) and a Keys matrix (size $d_k \times [K]$, represented by a green box with vertical lines). The result is then multiplied by a Values matrix (size $[V] \times d_v$, represented by a purple box with horizontal lines and an arrow pointing to one of them). The final output is a matrix of size $[Q] \times d_v$.

Blank Slide

II. (Weighted) Self Attention

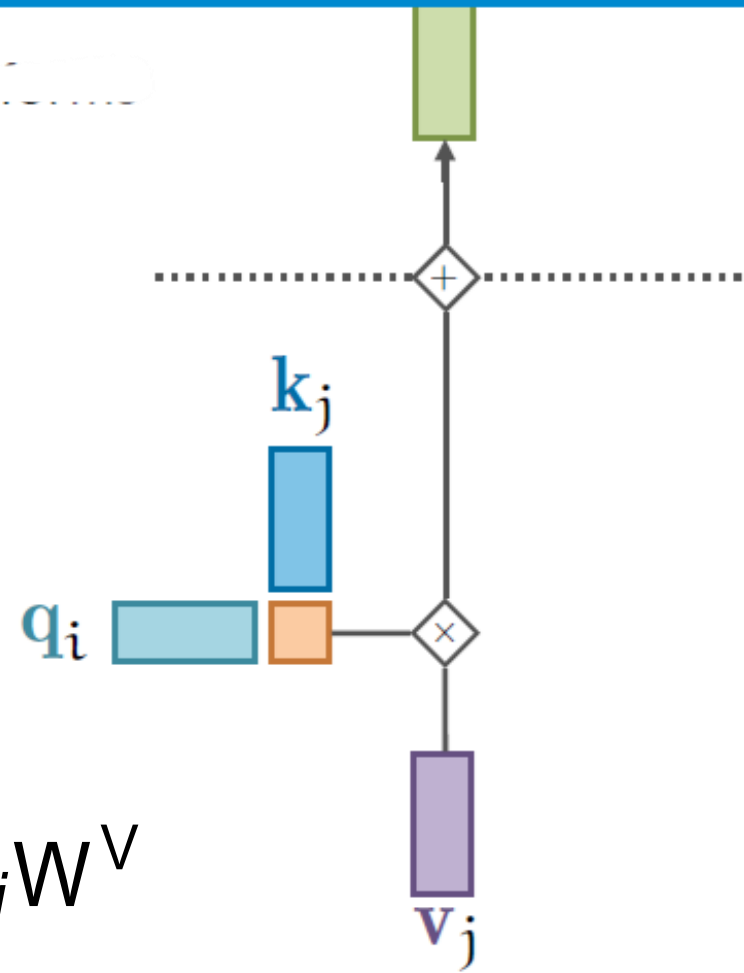
Learnable “Query”, “Key” and “Value”

$$\mathbf{k}_i = \mathbf{K}\mathbf{x}_i + \mathbf{b}_k$$

$$\mathbf{q}_i = \mathbf{Q}\mathbf{x}_i + \mathbf{b}_q$$

$$\mathbf{v}_i = \mathbf{V}\mathbf{x}_i + \mathbf{b}_v$$

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q; \mathbf{k}_i = \mathbf{x}_i \mathbf{W}^K; \mathbf{v}_i = \mathbf{x}_i \mathbf{W}^V$$



Blank Slide

Self Attention

One query per input vector

Inputs:

Input vectors: X (Shape: $N_X \times D_Q$)

Key matrix: W_K (Shape: $D_X \times D_Q$)

Value matrix: W_V (Shape: $D_X \times D_V$)

Query matrix: W_Q (Shape: $D_Q \times D_Q$)

Computation:

Query Vectors $Q = XW_Q$

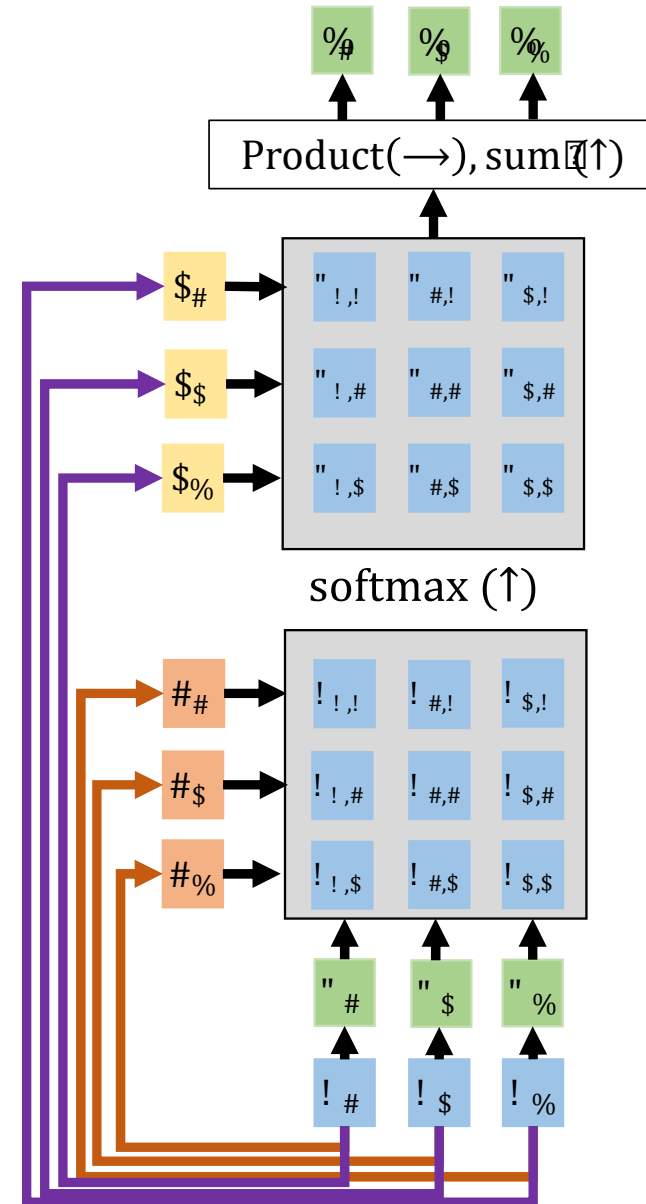
Key vectors: $K = XW_K$ (Shape: $N_X \times D_K$)

Value Vectors: $V = XW_V$ (Shape: $N_X \times D_V$)

Similarities: $E = \frac{Q \cdot K^T}{\sqrt{D_K}}$ (Shape: $N_X \times N_X$) $E_{\%} = (Q_{\%} \cdot K) / \sqrt{D_K}$

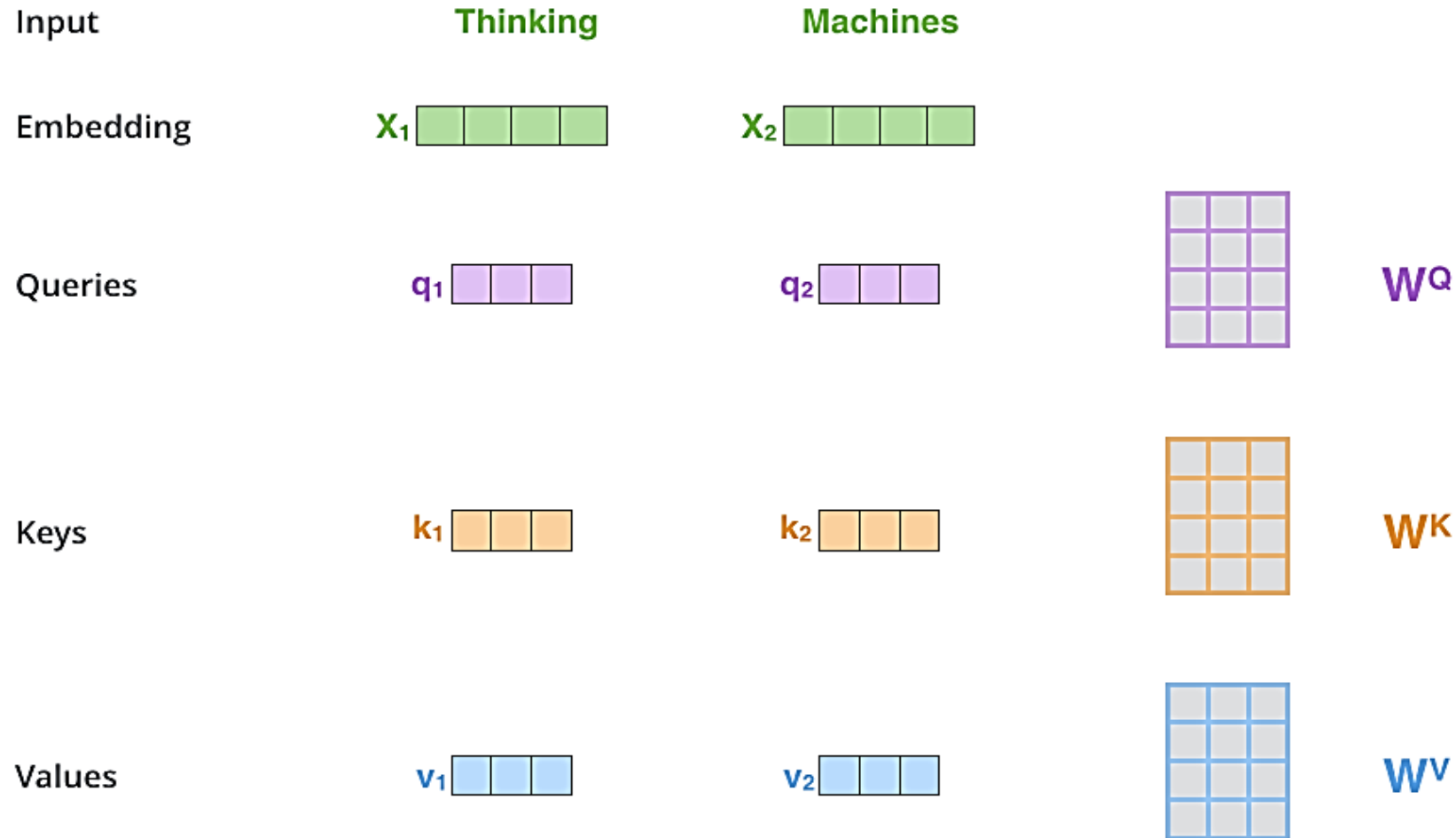
Attention weights: $A = \text{softmax}(E, \text{dim} = 1)$ (Shape: $N_X \times N_X$)

Output vectors: $Y = AV$ (Shape: $N_X \times D_V$) $Y_{\%} = \sum A_{\%} V$

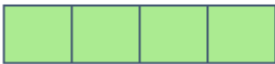
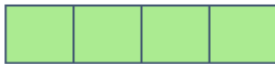
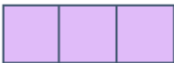
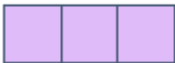
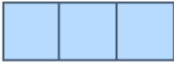
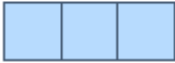


Blank Slide

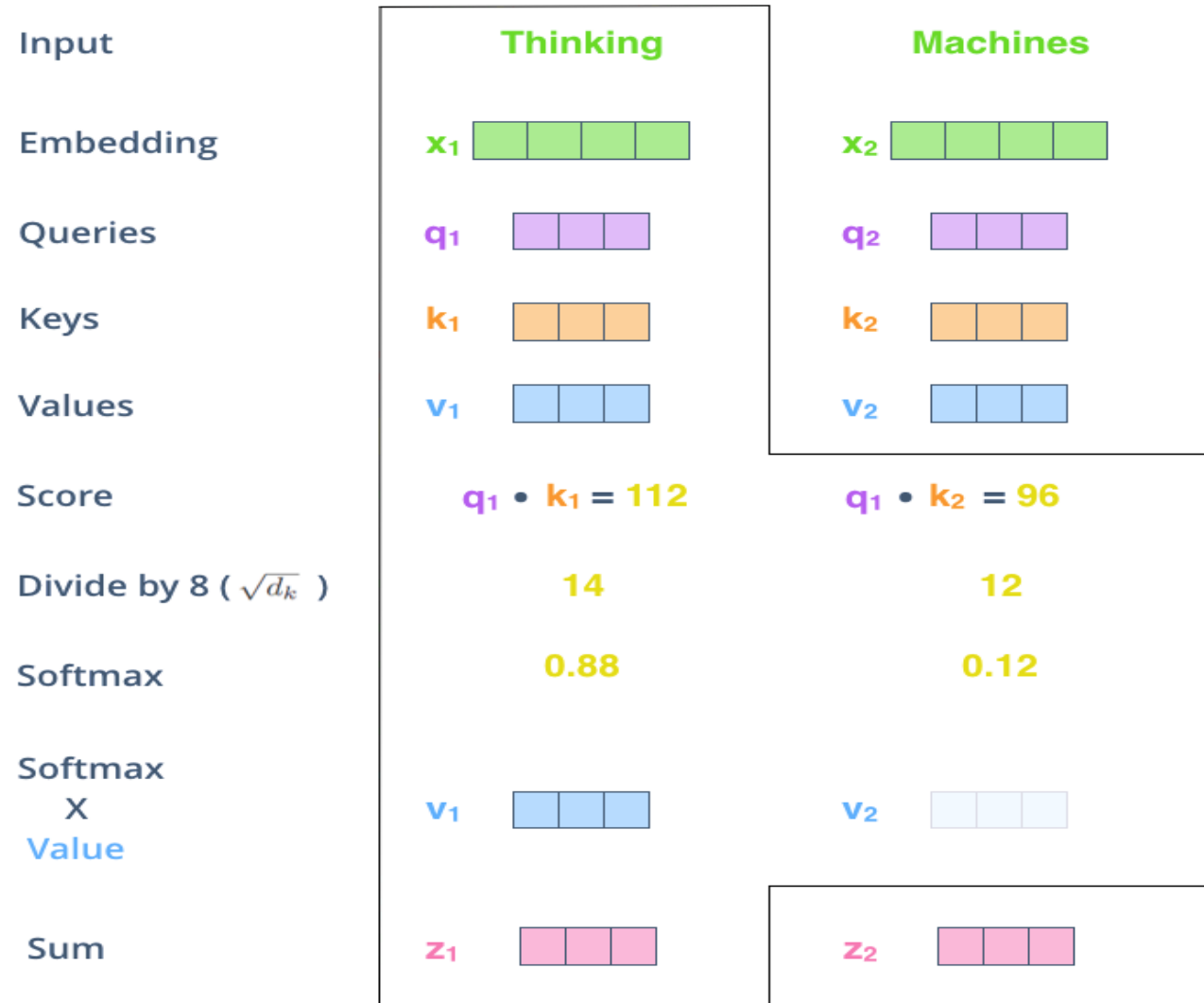
Self Attention: Step 1



Self Attention: Step 2

Input	Thinking	Machines
Embedding	x_1 	x_2 
Queries	q_1 	q_2 
Keys	k_1 	k_2 
Values	v_1 	v_2 
Score	$q_1 \cdot k_1 = 112$	$q_1 \cdot k_2 = 96$
Divide by 8 ($\sqrt{d_k}$)	14	12
Softmax	0.88	0.12

Self Attention: Step 3



Matrix View

$$\begin{array}{c} \text{X} \\ \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array} \end{array} \times \begin{array}{c} \text{W}^Q \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{array} = \begin{array}{c} \text{Q} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{array}$$

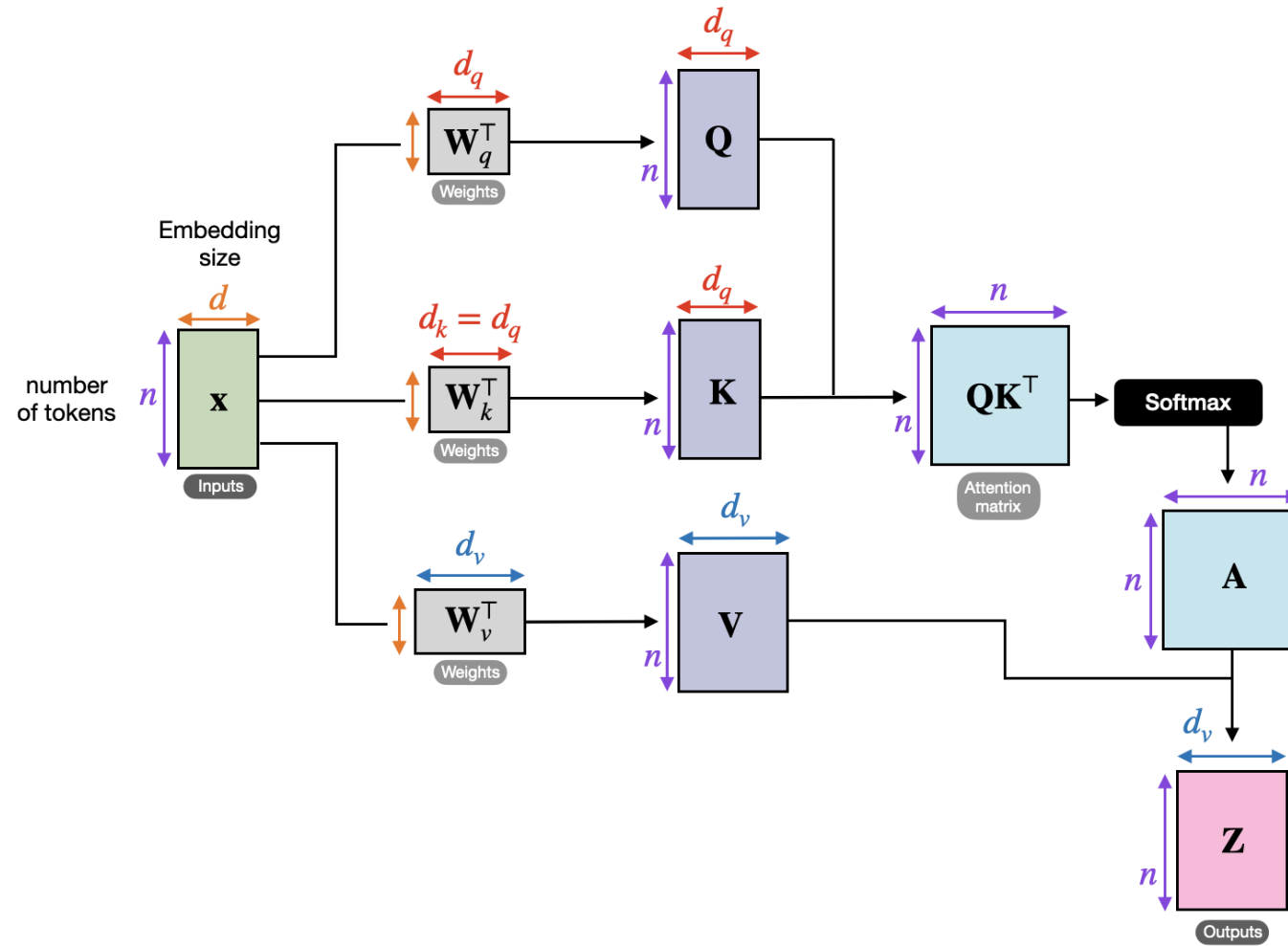
$$\begin{array}{c} \text{X} \\ \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array} \end{array} \times \begin{array}{c} \text{W}^K \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{array} = \begin{array}{c} \text{K} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{array}$$

$$\begin{array}{c} \text{X} \\ \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array} \end{array} \times \begin{array}{c} \text{W}^V \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{array} = \begin{array}{c} \text{V} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{array}$$

$$\begin{array}{c} \text{softmax} \left(\frac{\begin{array}{c} \text{Q} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{array} \times \begin{array}{c} \text{K}^T \\ \begin{array}{|c|} \hline \square \\ \hline \square \\ \hline \square \\ \hline \end{array} \end{array}}{\sqrt{d_k}} \right) \begin{array}{c} \text{V} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{array} \\ \\ = \begin{array}{c} \text{Z} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{array}
 \end{array}$$

Blank Slide

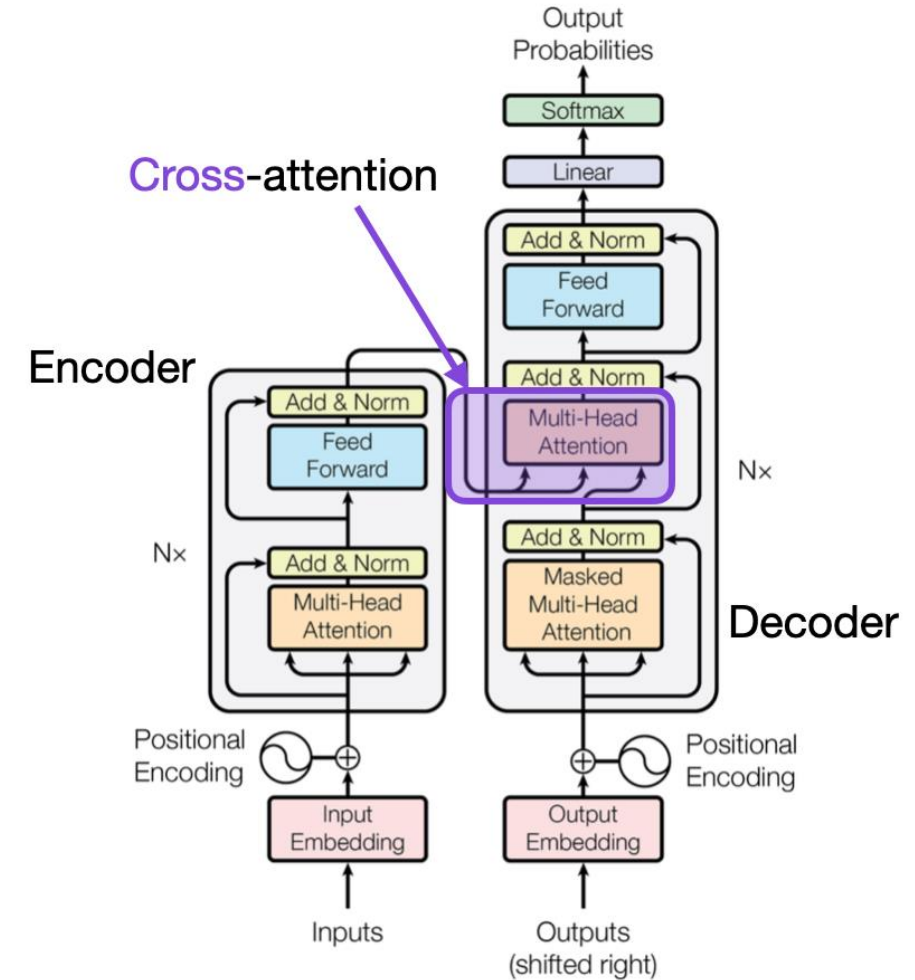
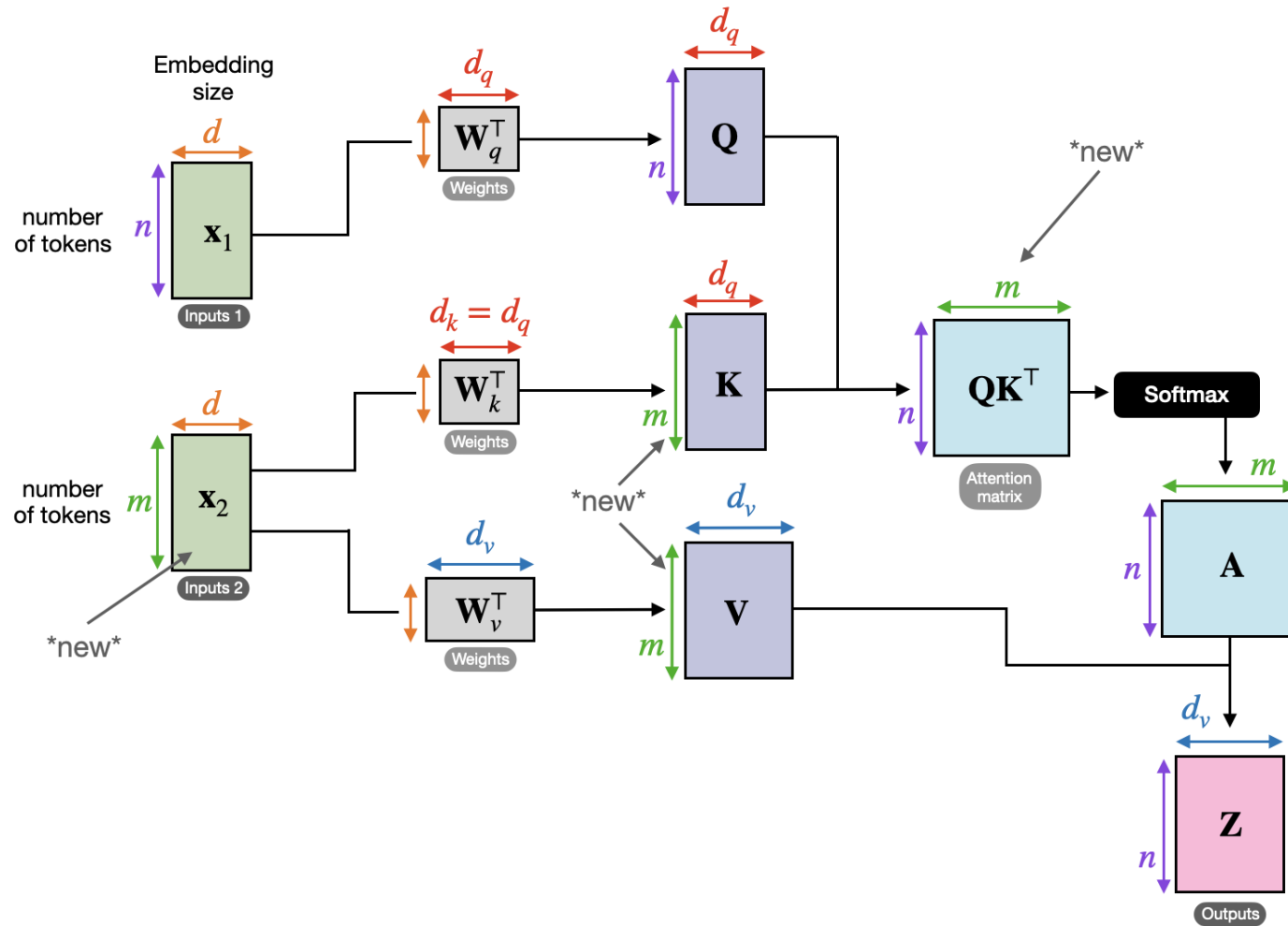
Self Attention: Computation



Example of Cross-Attention

- Task:
 - Given an Image (a set of patches), create a sentence
- Need of attention:
 - At any stage of creating a word, attend more to the region that is relevant

Cross Attention

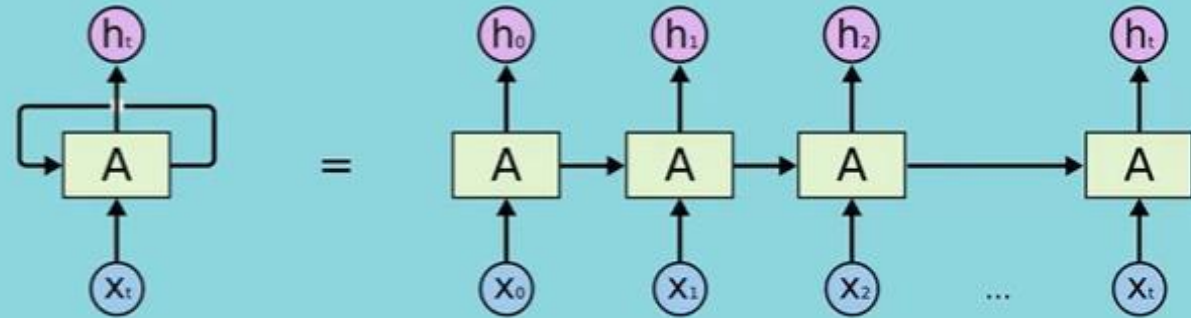
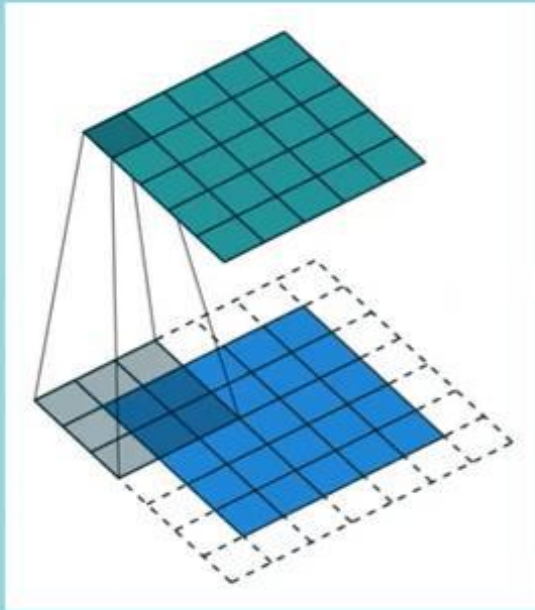


Source: "Attention Is All You Need" (<https://arxiv.org/abs/1706.03762>)

Example: Note that the queries usually come from the decoder, and the keys and values usually come from the encoder.

III. Discussions

CNN, RNN and Transformers



Pro: Compute in Parallel

Con: Finite Horizon

Pro: Infinite Horizon

Con: Compute Serially only

Attention

Visualization

Example: English to
French translation

Input: “The agreement on
the European Economic
Area was signed in
August 1992.”

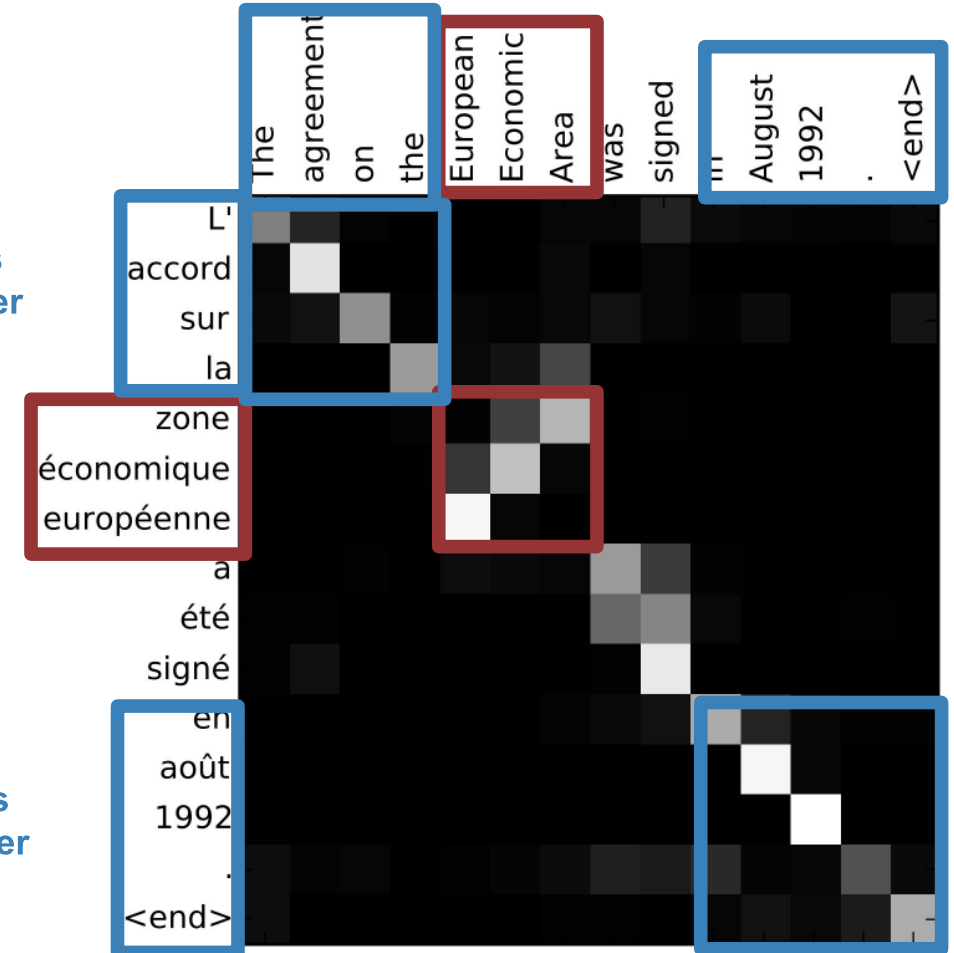
Output: “L'accord sur la
zone économique
européenne a été signé
en août 1992.”

Diagonal attention means
words correspond in order

Attention figures out
different word orders

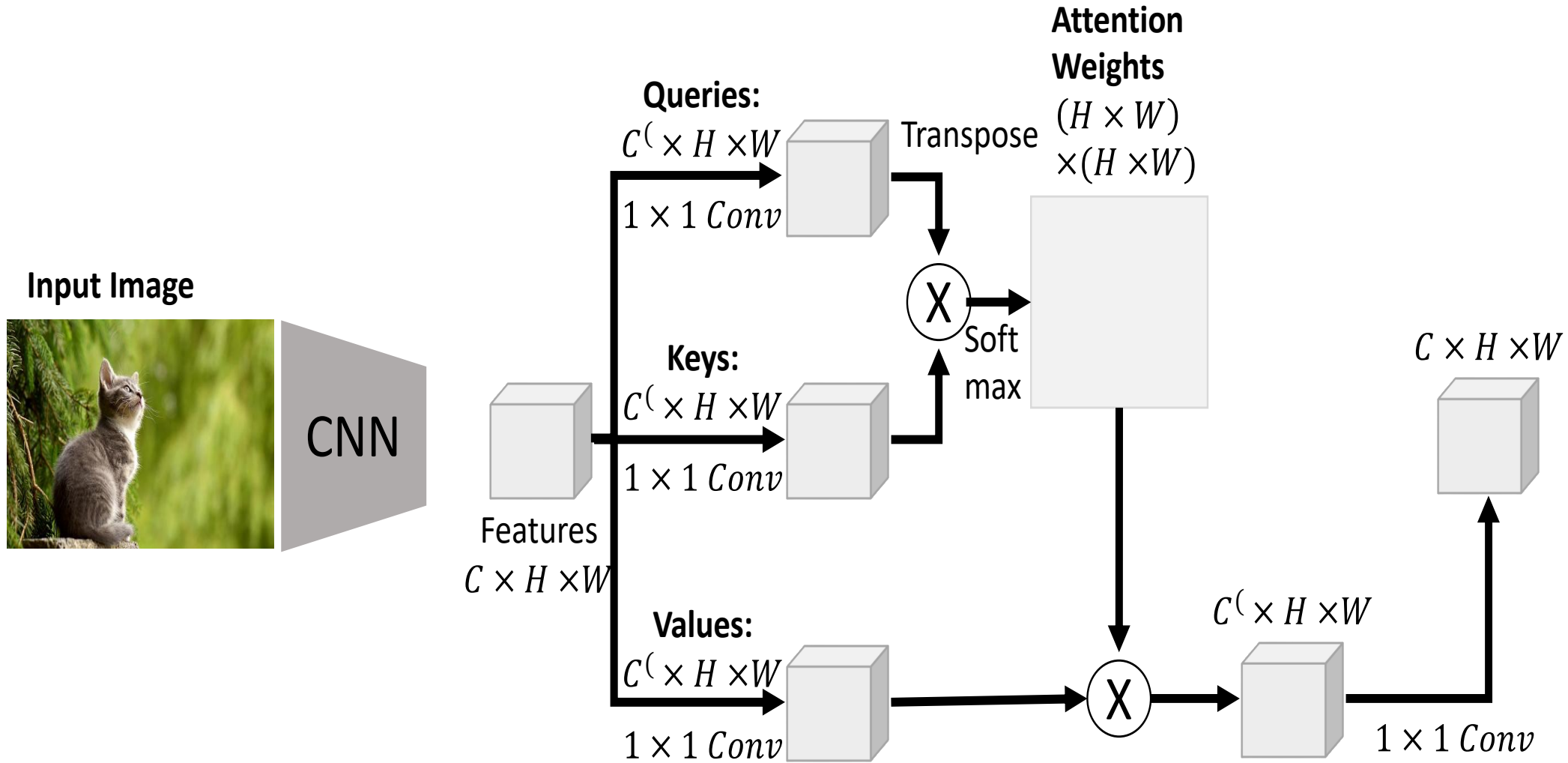
Diagonal attention means
words correspond in order

Visualize attention weights $a_{t,i}$



Blank Slide

CNN with Attention

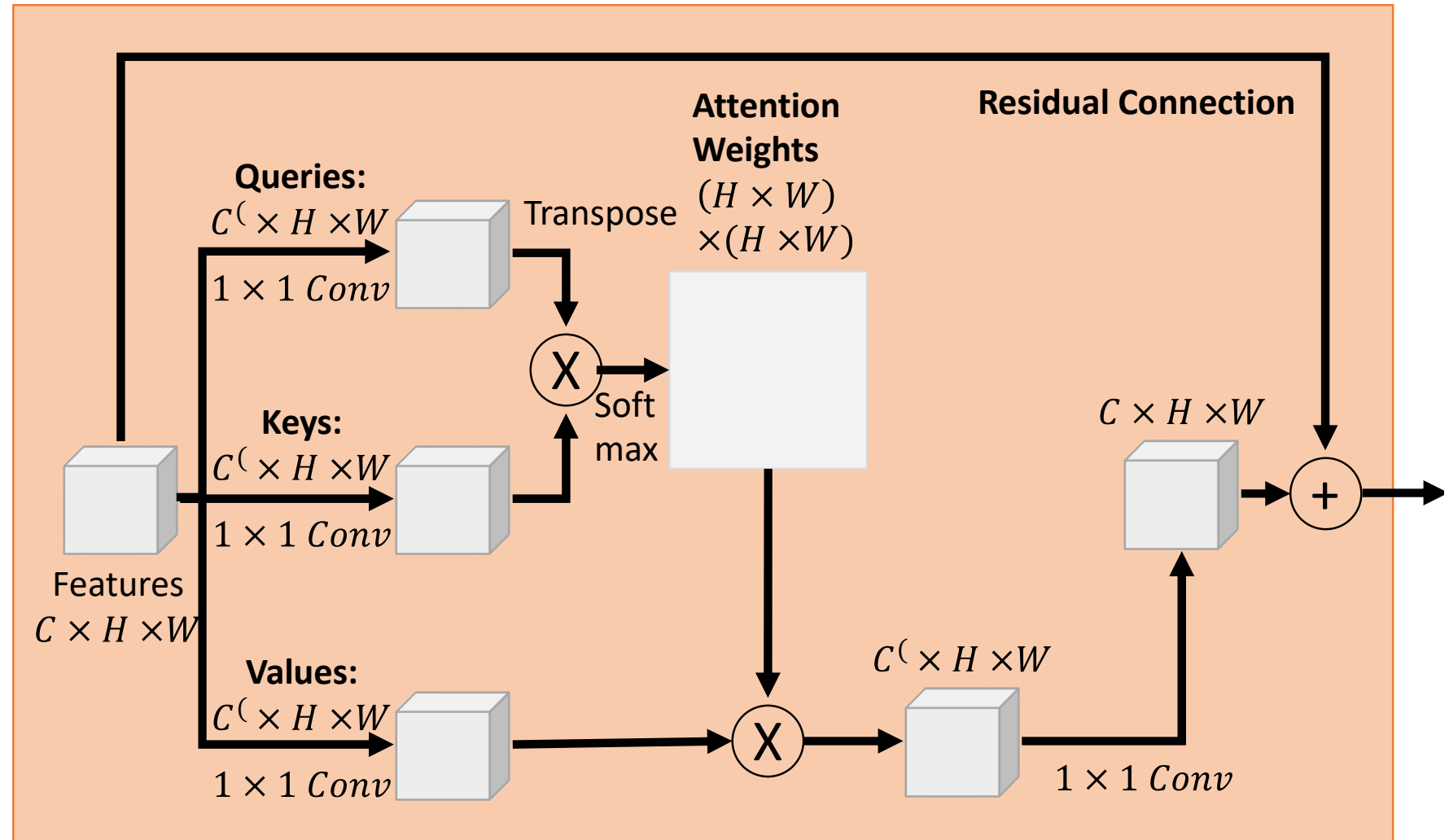


CNN with Attention

Input Image

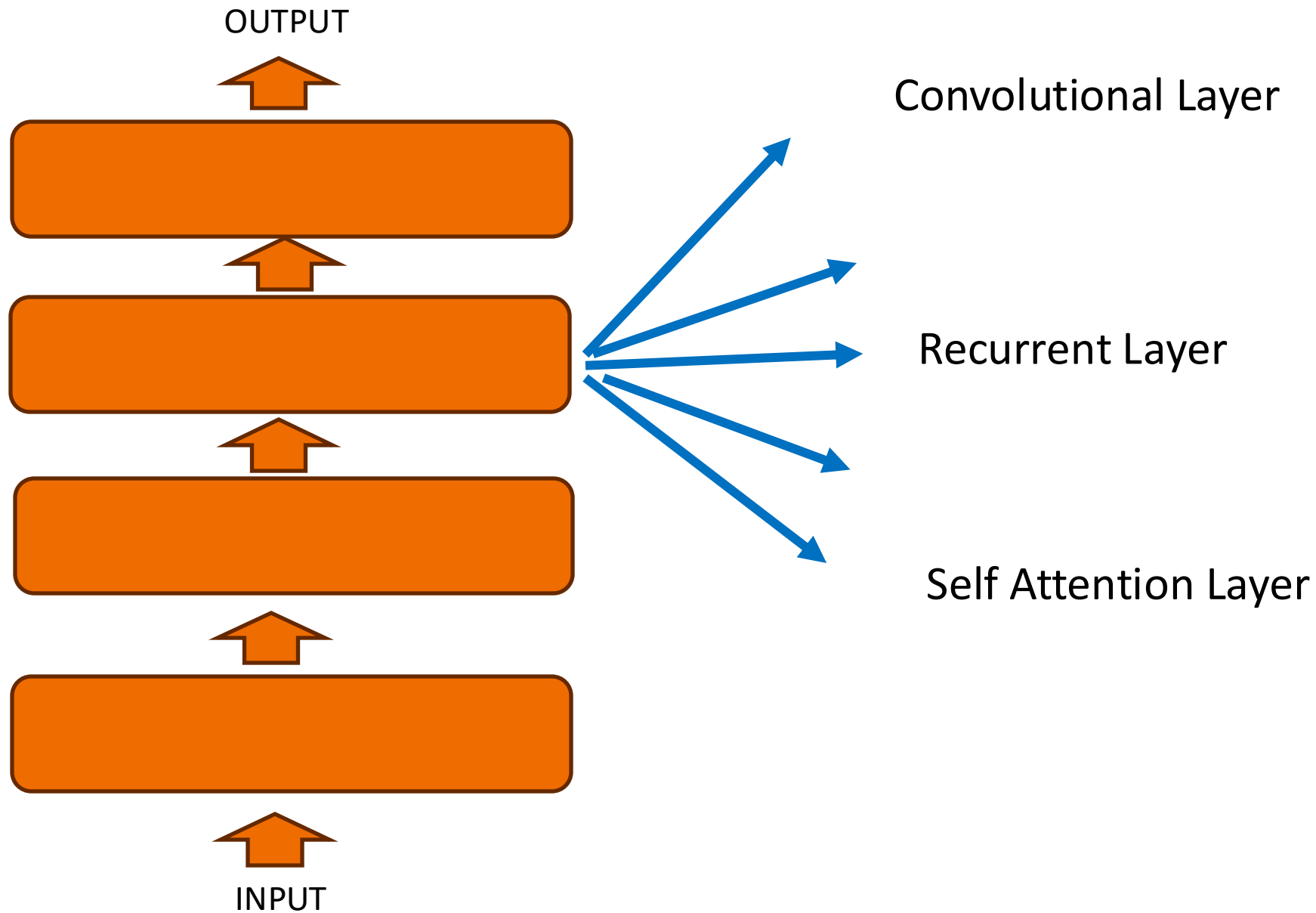


CNN



Self Attention Module

Self Attention Layer

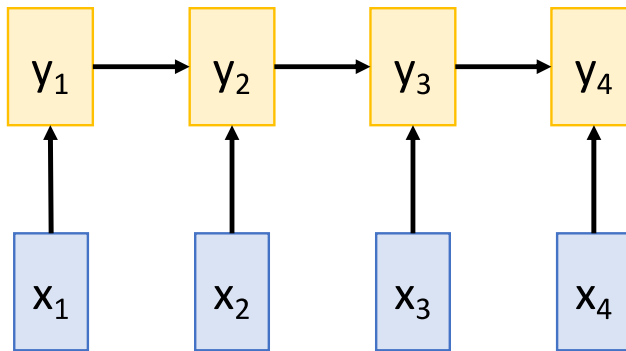


Self Attention: Comments

- Self attention learns the relationship between elements in a sequence.
 - say between words in a sentence
- Self Attention Vs Convolution
 - Filters are dynamically calculated instead of static filters
 - SA is invariant to changes in the input points
 - SA can operate on irregular inputs
- SA allows to learn global and local features
 - Hierarchical feature learning by cascading

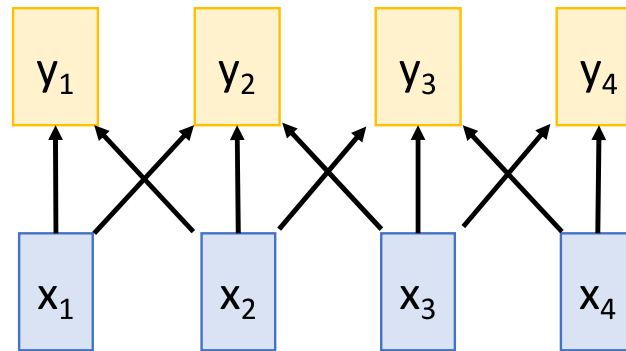
Conceptual Comparison

Recurrent Neural Network



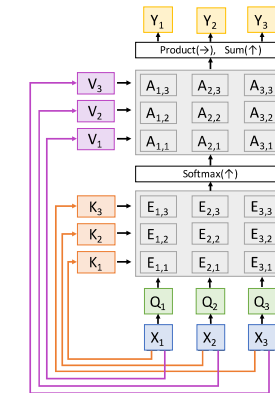
- Works on Ordered Sequences
- Long range dependence (in the past)
- Not parallelizable

1D Convolution



- Works on Multidimensional grids
- Short visibility; Stacking helps
- Highly parallelizable

Self-Attention



- Works on Sets and Sequences
- Long range dependence
- Parallelizable
- Memory Intensive

Thanks!!

Questions?